DeepSeek LLM Scaling Open-Source Language Models with Longtermism

1 研究背景：Scaling Law的争议与深化

DeepSeek团队开篇就指出了LLM领域的“一朵乌云”：虽然大家都认可Scaling Law的存在，但不同研究得出的**最优模型规模与数据规模的配比**却大相径庭。

- **经典争议**：
  - **OpenAI 路线**：倾向于训练**更大的模型**，使用相对较少的数据（如GPT-3）。
  - **Chinchilla 路线**：通过著名的Chinchilla定律指出，当前许多大模型是“训练不足”的，在相同算力下，应该训练**小一些的模型，但使用更多的数据**。
- **DeepSeek的洞察**：这些分歧可能源于**实验设置、超参数选择、以及模型规模的定义不统一**。如果无法精确掌握Scaling Law，那么在规划耗资巨大的大模型训练时，就会像没有地图的航行，风险极高。因此，他们的研究旨在**消除这些不确定性，为后续的模型扩展提供一张可靠的“地图”**。

2 模型架构的深层设计逻辑

DeepSeek LLM的架构在遵循LLaMa的基础上，做出了一个关键性的选择：

- **核心创新：深度优先的缩放策略**
  - **常规做法**：在缩放模型时，通常同步增加模型的**深度**（层数）和**宽度**（特别是FFN层的隐藏维度）。
  - **DeepSeek的选择**：对于67B模型，他们**显著增加了网络深度（达到95层）**，而没有同比例地增加FFN的宽度。
  - **理论依据**：
    1. **更强的表征能力**：更深的网络可能学习到更复杂、更层次化的特征表示。
    2. **效率考量**：在某些硬件架构上，增加深度可能比增加宽度在计算和通信上更高效。
    3. **探索最优解**：他们希望通过实验验证，在Scaling Law的框架下，这种“深度优先”的策略是否能带来比传统策略更优的性能。这本身就是对Scaling Law的一种实践和验证。

3 超参数Scaling Law的深度剖析

1. 问题根源：

早期工作（如OpenAI的Scaling Law论文）给出了超参数设置的经验法则，但DeepSeek在初步实验中发现，这些法则在他们自己的实验设置下“参考价值有限”。这意味着**超参数的最优值强烈依赖于具体的数据集、模型架构和训练环境**，不能直接照搬。

2. 方法论：

- **控制变量**：他们固定了所有影响较小的超参数，只聚焦于对训练影响最大的两个：**全局批量大小** 和 **学习率**。
- **网格搜索**：首先在小的算力规模（177M FLOPs/token）上进行广泛的网格搜索，确立了一个基准。
- **拟合定律**：在多个算力规模（从 $1e17$ 到 $2e19$ ）上训练模型，并找出那些“泛化误差不超过最小值0.25%”的**近最优超参数集合**。然后，他们用幂律函数来拟合 B_{opt} 和 η_{opt} 与算力规模 C 的关系。

3. 关键发现与公式：

$$\eta_{opt} = 0.3118 \cdot C^{-0.1250} \quad , \quad B_{opt} = 0.2920 \cdot C^{0.3271}$$

- **物理意义**:
  - 学习率随着算力增加而缓慢下降（$C^{-0.125}$），这是因为大规模训练需要更谨慎的更新步伐以避免震荡。
  - 批量大小随着算力增加而增长（$C^{0.327}$），这符合通用理论：更大的算力允许我们使用更大的批量以充分利用并行性，而不会因泛化误差上升而受损。
- **"宽范围"的启示**：他们发现近最优超参数存在于一个**较宽的范围内**，这是一个非常实用的发现。它意味着在实际训练中，我们不需要追求一个绝对的最优单点，只要落在这个"高原"区域，就能获得近乎最优的性能。这降低了大规模调参的工程难度。

4 模型规模度量的革命：从 N 到 M

这是DeepSeek论文中最具洞察力的贡献之一，它解决了Scaling Law研究中的一个根本性缺陷。

1. 传统度量的不精确性：

- $6N_1$：只考虑了Transformer层中MatMul的FLOPs，完全忽略了**注意力机制** 的计算量。
- $6N_2$：在 $6N_1$ 基础上加上了词嵌入层的计算，但这部分在推理时是**一次性查找操作**，其计算开销与模型的前向传播FLOPs无关，加入它反而会**稀释和扭曲**模型核心计算能力的表征。

2. 新度量 M（非嵌入层 FLOPs/token）的优越性：

$$M = 72n_{layer}d_{model}^2 + 12n_{layer}d_{model}l_{seq}$$

- **第一部分** $72n_{layer}d_{model}^2$：对应于QKV投影、FFN上/下投影等密集矩阵乘法操作。这是模型计算的主体。
- **第二部分** $12n_{layer}d_{model}l_{seq}$：**关键补充！** 这代表了注意力机制中Q · K^T 和 Score · V 矩阵乘法的计算量。它正比于序列长度 l_{seq}，而这一部分在以前的度量中被完全忽略了。

5 数据分配Scaling Law与数据质量的关联

1. 实验设计：

在确立了精确的模型度量 M 和超参数设置方法后，他们使用**IsoFLOPs** 方法进行实验：为多个固定的算力预算 C，分别训练一系列具有不同 M (模型规模) 和 D (数据规模) 组合的模型，然后绘制出每个预算下的损失曲线，曲线的最低点即为该预算下的最优 M/D 配比。

2. 核心拟合结果：

$$M_{opt} = 0.1715 \cdot C^{0.5243} \quad , \quad D_{opt} = 5.8316 \cdot C^{0.4757}$$

解读：最优模型规模的指数（0.524）略大于最优数据规模的指数（0.476）。这表明，在DeepSeek的设置下，**将额外的算力稍微倾向于放大模型，比倾向于增加数据更有效**。这个结论介于OpenAI和Chinchilla之间，但更接近Chinchilla，强调了数据的重要性，同时也没有忽视模型规模的增长。

3. 突破性发现：数据质量决定缩放策略

这是论文中最引人入胜的发现之一。DeepSeek团队在三个不同质量的数据集上重复了Scaling Law实验：

结果：他们观察到，**随着数据质量的提升，最优的模型缩放指数 a 会增大，而数据缩放指数 b 会减小**。

深层解读:

- **高质量数据是"高效教材"**：高质量数据信息密度高、噪声少、逻辑清晰。模型可以更高效地从每个token中

学习到知识。因此，当数据质量很高时，**增加模型容量（M）** 以学习数据中更复杂、更微妙的模式，其收益要大于简单地投入更多数据（D）。因为模型有能力消化这些“精华”，而不会轻易过拟合。

- **低质量数据是“题海战术”**：低质量数据包含大量冗余和噪声。模型需要“刷”更多的题（token）才能偶然学到有用的知识。此时，盲目增大模型容量可能导致模型记住噪声，而使用一个稍小但训练更充分的模型（更多数据）可能是更稳健的策略。
- **统一了争议**：这一发现巧妙地解释了为什么OpenAI和DeepMind得出了不同的Scaling Law结论——**他们很可能使用了不同质量的数据集**。OpenAI可能拥有质量极高的私有数据，因此倾向于建更大的模型；而Chinchilla研究基于相对通用的公开数据，结论则偏向于使用更多数据。

DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models

1 背景与动机

在大语言模型（LLM）时代，**混合专家**（MoE）架构因其能够在扩大模型参数规模的同时有效控制计算开销，成为一种主流范式。然而，传统 MoE 架构存在两个关键瓶颈：

1. **知识混合**（Knowledge Mixing）
   - 专家数量有限（如 8–16 个），导致单个专家需处理多种类型的知识，难以专业化。
2. **知识冗余**（Knowledge Redundancy）
   - 多个专家重复学习通用知识，造成参数浪费，降低效率。

这些问题限制了 MoE 模型逼近其理论性能上限的能力——即等效稠密模型的性能。

2 DeepSeekMoE 的核心创新

2.1 细粒度专家分割（Fine-grained Expert Splitting）

- **方法**：将每个标准 FFN 专家沿中间隐藏层维度切分为 m 个更小的子专家，总专家数从 N 扩展至 mN。
- **激活机制**：同时将 top-K 路由扩展为 top-mK，保持总计算量不变。
- **优势**：
  - 知识解耦更精细，不同子专家可专注特定知识子集；
  - 激活组合数量剧增（例如从 $\binom{16}{2} = 120$ 到 $\binom{64}{8} \approx 4.4 \times 10^9$），提升路由灵活性与表达能力。

公式表达（细粒度 MoE 层输出）：

$$\mathbf{h}_t^l = \sum_{i=1}^{mN} g_{i,t} \cdot \mathrm{FFN}_i(\mathbf{u}_t^l) + \mathbf{u}_t^l$$

其中仅 top-mK 个门控值 $g_{i,t}$ 非零。

2.2 共享专家隔离（Shared Expert Isolation）

- **方法**：从 mN 个专家中固定 K_s 个作为**共享专家**，对所有 token 始终激活。
- **目的**：集中承载通用知识，避免路由专家重复学习共性内容。
- **调整机制**：路由专家激活数相应减少为 $mK - K_s$，维持总计算量恒定。

公式表达（含共享专家）：

$$\mathbf{h}_t^l = \underbrace{\sum_{i=1}^{K_s} \mathrm{FFN}_i(\mathbf{u}_t^l)}_{\text{共享专家}} + \underbrace{\sum_{i=K_s+1}^{mN} g_{i,t} \cdot \mathrm{FFN}_i(\mathbf{u}_t^l)}_{\text{路由专家}} + \mathbf{u}_t^l$$

其中路由部分仅激活 top-$(mK - K_s)$ 个专家。

3 负载均衡设计

3.1 专家级均衡损失（Expert-level Balance）

- 防止“路由坍缩”(少数专家被过度使用)。
- 定义:

$$\mathcal{L}_{\text{ExpBal}} = \alpha_1 \sum_{i=1}^{N'} f_i P_i$$

  - f_i：专家 i 被选中的频率;
  - P_i：专家 i 的平均门控分数;
  - $N' = mN - K_s$，$K' = mK - K_s$。

3.2 设备级均衡损失（Device-level Balance）

- 优化跨设备计算负载，避免通信或算力瓶颈。
- 将专家分组到 D 个设备，定义:

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{d=1}^{D} f'_d P'_d$$

  - f'_d, P'_d 为设备 d 上专家的平均频率与门控分数。

实践中：α_1 设为较小值（防坍缩），α_2 设为较大值（保设备均衡）。

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

1 研究动机与问题深析

1.1 数学推理的独特挑战

数学推理不同于一般的文本生成或代码生成，它要求模型：

- **精确性**：每一步推导都必须符合严格的数学逻辑，容错率极低。
- **符号抽象**：需要理解并操纵复杂的数学符号系统。
- **多步规划**：解决一个问题往往需要一连串逻辑严密的步骤，任何一步出错都可能导致满盘皆输。
- **知识融合**：需要将领域知识（如定理、公式）与逻辑推理能力有机结合。

1.2 现有方法的局限性

- **闭源模型主导**：GPT-4、Gemini-Ultra等表现出色，但透明度低，无法深入研究其技术细节。
- **开源模型落后**：在DeepSeekMath之前，最好的开源模型在MATH数据集上也只能达到30%左右，与闭源模型差距显著。
- **规模依赖误区**：普遍认为提升数学能力主要依靠扩大模型规模，但Minerva 540B的表现证明，单纯增大参数并非最优解。

2 数据工程的深度剖析

2.1 数据筛选管线的技术细节

DeepSeekMath的数据构建过程是一个精心设计的迭代优化系统：

第一阶段：初始分类器训练

- **正样本**：来自OpenWebMath的高质量数学网页。
- **负样本**：不仅包括随机网页，还特意加入了可能与数学混淆的负样本（如包含大量数字的金融新闻、体育统计等），以提高分类器的辨别力。
- **分类器选择**：使用fastText而非更深度的模型，是在效果和处理海量数据的效率之间做出的权衡。

第二阶段：数据挖掘与人工精标

- **规模化挖掘**：使用训练好的分类器处理Common Crawl，获得候选数学文档。
- **质量评估**：人工标注者根据一套严格的标准评判网页质量，标准包括：
  - **数学内容密度**：页面中实质性数学内容的比例。
  - **内容准确性**：数学内容是否正确无误。
  - **表述清晰度**：数学概念的阐述是否清晰、有条理。
- **构建黄金数据集**：通过人工精标，形成了一个高质量的“种子”数据集。

第三阶段：分类器迭代

- 使用人工精标后的高质量数据重新训练分类器。
- 迭代执行第二、三步，不断放大高质量数据的规模并提升分类器性能。
- 最终得到的DeepSeek-Math数据集规模达到120B tokens，是Minerva数据集的7倍，且质量有保障。

2.2 代码与数学的协同效应

DeepSeek选择从DeepSeek-Coder作为起点，是基于深刻的洞察：

- **结构化思维**：编程训练了模型的结构化、逻辑化思维能力，这与数学推理所需的思维方式高度一致。
- **符号操作**：代码中的变量、函数、流程控制与数学中的符号演算有内在的相似性。
- **工具使用基础**：许多数学问题可以通过生成程序代码来解决（如计算几何、组合数学问题），代码预训练为这种“程序思维”求解方式奠定了基础。
- **实验验证**：论文中通过消融实验证明，从代码模型开始比从通用语言模型开始，最终数学性能高出约5-8个百分点。

3 GRPO算法的技术深度解析

3.1 PPO在LLM微调中的固有缺陷

要理解GRPO的创新，首先要明白PPO的问题：

- **价值网络的不稳定性**：在LLM场景中，奖励通常只在序列末尾给出，导致价值网络训练信号稀疏且嘈杂。
- **高内存消耗**：策略网络πθ和价值网络Vψ通常参数量相当，同时训练意味着内存占用几乎翻倍。
- **复杂度高**：需要精心平衡策略损失、价值损失和熵奖励等多个目标函数。

3.2 GRPO的统计原理

GRPO的核心是用**组内统计量**替代价值网络：

优势估计的数学基础：

- 在策略梯度中，基线（baseline）的功能是减少方差，而不引入偏差。
- 理论上，任何与动作无关的基线都是无偏的。一组样本的均值是满足这一条件的有效选择。
- 通过减去组均值、除以组标准差，GRPO实现了**奖励的标准化**，这使得：
  - 不同问题间的奖励尺度差异被消除。
  - 优化过程更加稳定。
  - 模型专注于在**同一问题的不同解答间**进行相对比较。

与PPO的目标函数对比：

- **PPO目标**：

$$\mathcal{J}_{PPO} = \mathbb{E}[\min(ratio \cdot A_t, clip(ratio, 1-\epsilon, 1+\epsilon) \cdot A_t)]$$

  其中优势A_t通过GAE计算，依赖价值网络Vψ。
- **GRPO目标**：

$$\mathcal{J}_{GRPO} = \mathbb{E}[\min(ratio \cdot \tilde{r}_i, clip(ratio, 1-\epsilon, 1+\epsilon) \cdot \tilde{r}_i) - \beta D_{KL}]$$

  其中$\tilde{r}_i$是归一化的组内奖励。

3.3 KL散度估计的创新

GRPO中使用了KL散度的无偏估计器：

$$\hat{D}_{KL} = \frac{\pi_{ref}(a|s)}{\pi_\theta(a|s)} - \log\frac{\pi_{ref}(a|s)}{\pi_\theta(a|s)} - 1$$

这种估计方式确保了KL散度始终为非负值，避免了在训练初期因概率比值不稳定导致的训练崩溃。

3.4 过程监督的工程实现

论文中对过程监督的实现值得关注：

- **奖励分配**：不仅在最终答案处给予奖励，在每个推理步骤（如每个等式变换后）都提供即时奖励。
- **优势计算**：使用**折扣累积奖励**$\hat{A}_{i,t} = \sum_{j \geq t} \gamma^{j-t} \tilde{r}_i^{index(j)}$，其中γ是折扣因子。
- **效果**：这种细粒度的奖励信号引导模型学习“中间验证”的推理习惯，类似于人类在解题时检查每一步是否正确。

4 实验设计与分析的深度洞察

4.1 开放领域泛化的机制分析

一个有趣的发现是：尽管RL训练只使用了GSM8K和MATH的英文数据，模型在中文数学基准(CMATH)和其他开放领域任务上也表现出提升。这暗示：

- **RL不仅仅是记忆**：它可能增强了模型的**推理机制**本身，而不仅仅是优化了对特定类型问题的应答。
- **泛化路径**：通过强化学习，模型学会了“如何更好地思考”，这种能力可以迁移到不同语言和稍有问题域的任务中。

4.2 统一范式的理论价值

DeepSeek提出的统一范式将RFT、DPO、PPO、GRPO等方法放在同一个框架下理解：

- **RFT**可视为隐式的策略优化，通过从高质量样本中重训练来实现。
- **DPO**是通过解析式优化偏好目标，避免了显式的奖励建模。
- **PPO**是标准的Actor-Critic在线强化学习。
- **GRPO**是PPO的高效变种，用组统计替代Critic。

这个统一视角揭示了：所有这些方法本质上都是在解决**如何从模型自身生成的内容中识别并强化高质量推理路径**这一共同问题。

DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model

1 背景与问题深度分析

1.1 Transformer推理瓶颈的本质

传统Transformer模型在自回归生成过程中，需要缓存所有先前token的Key和Value向量，形成KV缓存。对于具有L层、h个头、每个头维度为d_k的模型，序列长度为n时，KV缓存的总大小为：

$$\text{缓存大小} = 2 \times L \times h \times d_k \times n$$

这导致：

- **内存瓶颈**：长序列生成时缓存占用巨大
- **计算瓶颈**：注意力复杂度为$O(n^2 d)$
- **吞吐量限制**：批量大小受限于缓存容量

1.2 现有压缩技术的局限性

MQA (Multi-Query Attention)：

- 所有注意力头共享同一Key和Value投影
- 缓存减少到：$2 \times L \times d_k \times n$
- 但性能损失显著，信息容量受限

GQA (Grouped-Query Attention)：

- 折中方案，头分组共享KV投影
- 仍无法完全保持MHA的表达能力

2 MLA核心机制深度剖析

2.1 低秩压缩的数学基础

MLA基于矩阵低秩近似的理论基础。注意力矩阵本质上是低秩的，这为压缩提供了理论保证。

低秩分解原理：

给定矩阵$A \in \mathbb{R}^{m \times n}$，寻找秩$r$矩阵$A_r$使得：

$$\min_{rank(A_r)=r} \|A - A_r\|_F$$

2.2 MLA完整计算图

2.2.1 输入投影阶段

$$\begin{aligned}
\mathbf{h}_t &\in \mathbb{R}^d \quad \text{// 输入隐状态} \\
\mathbf{c}_t^{KV} &= W^{DKV}\mathbf{h}_t \quad \text{// KV联合压缩} \\
\mathbf{c}_t^{Q} &= W^{DQ}\mathbf{h}_t \quad \text{// 查询压缩}
\end{aligned}$$

其中：

- $W^{DKV} \in \mathbb{R}^{d_c \times d}$，$d_c \ll d$

- $W^{DQ} \in \mathbb{R}^{d'_c \times d}$，$d'_c \ll d$

2.2.2 注意力头分解

$$
\begin{aligned}
\mathbf{q}_t^C &= W^{UQ}\mathbf{c}_t^Q = [\mathbf{q}_{t,1}^C; \ldots; \mathbf{q}_{t,n_h}^C] \\
\mathbf{k}_t^C &= W^{UK}\mathbf{c}_t^{KV} = [\mathbf{k}_{t,1}^C; \ldots; \mathbf{k}_{t,n_h}^C] \\
\mathbf{v}_t^C &= W^{UV}\mathbf{c}_t^{KV} = [\mathbf{v}_{t,1}^C; \ldots; \mathbf{v}_{t,n_h}^C]
\end{aligned}
$$

2.3 解耦RoPE的详细实现

2.3.1 兼容性问题分析

原始方案中，如果对压缩后的Key应用RoPE：

$$
\mathbf{k}_t^C = \mathrm{RoPE}(W^{UK}\mathbf{c}_t^{KV})
$$

导致W^{UK}与位置相关的旋转矩阵耦合，无法在推理时被吸收。

2.3.2 解耦方案设计

$$
\begin{aligned}
\mathbf{q}_t^R &= \mathrm{RoPE}(W^{QR}\mathbf{c}_t^Q) = [\mathbf{q}_{t,1}^R; \ldots; \mathbf{q}_{t,n_h}^R] \\
\mathbf{k}_t^R &= \mathrm{RoPE}(W^{KR}\mathbf{h}_t) \\
\mathbf{q}_{t,i} &= [\mathbf{q}_{t,i}^C; \mathbf{q}_{t,i}^R] \\
\mathbf{k}_{t,i} &= [\mathbf{k}_{t,i}^C; \mathbf{k}_t^R]
\end{aligned}
$$

3 推理优化技术细节

3.1 矩阵吸收的数学证明

Key投影吸收：

$$
\begin{aligned}
\mathbf{q}_{t,i}^T\mathbf{k}_{j,i}^C &= (W_i^{UQ}\mathbf{c}_t^Q)^T(W_i^{UK}\mathbf{c}_j^{KV}) \\
&= (\mathbf{c}_t^Q)^T(W_i^{UQ})^T W_i^{UK}\mathbf{c}_j^{KV} \\
&= (\mathbf{c}_t^Q)^T W_i^{QK}\mathbf{c}_j^{KV}
\end{aligned}
$$

其中$W_i^{QK} = (W_i^{UQ})^T W_i^{UK} \in \mathbb{R}^{d'_c \times d_c}$可预先计算。

Value投影吸收：

$$
\begin{aligned}
\mathbf{o}_t &= \sum_{j=1}^{t} \alpha_{t,j}\mathbf{v}_j^C \\
&= \sum_{j=1}^{t} \alpha_{t,j} W^{UV}\mathbf{c}_j^{KV} \\
&= W^{UV}\sum_{j=1}^{t} \alpha_{t,j}\mathbf{c}_j^{KV}
\end{aligned}
$$

输出投影可合并：

$$
\mathbf{u}_t = W^O\mathbf{o}_t = (W^O W^{UV})\sum_{j=1}^{t} \alpha_{t,j}\mathbf{c}_j^{KV}
$$

3.2 缓存机制优化

MLA缓存需求:

$$\text{缓存大小} = L \times (d_c + d_h^R) \times n$$

与传统方法对比:

- MHA：$2 \times L \times h \times d_k \times n$
- MQA：$2 \times L \times d_k \times n$
- MLA：$L \times (d_c + d_h^R) \times n$

典型配置下压缩比可达5-10倍。

DeepSeek-V3 Technical Report

1 用于负载均衡的辅助无损策略

1.1 问题背景与挑战

在混合专家（MoE）模型中，**专家负载不均衡**是一个关键挑战：

- **路由崩塌**：某些专家被过度使用，而其他专家处于闲置状态
- **计算效率下降**：在专家并行架构下，负载不均衡会导致计算资源浪费
- **传统方法的局限**：使用辅助损失函数来平衡负载，但过大的辅助损失会损害模型性能

1.2 辅助无损策略的核心创新

DeepSeek团队提出了**动态偏置调整策略**，在保持模型性能的同时实现负载均衡：

1.2.1 偏置项机制

- 为每个专家引入可训练的偏置项 b_i
- 在路由决策时，将原始亲和力分数 $s_{i,t}$ 与偏置项结合：

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \mathrm{Topk}(\{s_{j,t} + b_j | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

关键设计：

- 偏置项仅用于路由决策，不影响门控值计算
- 门控值仍使用原始亲和力分数 $s_{i,t}$，确保前向传播不受影响

1.2.2 动态调整过程

训练过程中的实时监控与调整：

- **持续监控**：在每个训练步结束时，统计整个批次的专家负载情况
- **偏置更新规则**：
  - 过载专家：偏置项减少 γ
  - 欠载专家：偏置项增加 γ
- **超参数**：γ 为偏置更新速度，控制调整幅度

1.3 互补序列辅助损失

为防止单个序列内的极端失衡，额外引入**互补序列辅助损失**：

1.3.1 损失函数构成

$$\mathcal{L}_{\mathrm{Bal}} = \alpha \sum_{i=1}^{N_r} f_i P_i$$

其中：

- $f_i = \frac{N_r}{K_r T} \sum_{t=1}^{T} \mathbb{1}(s_{i,t} \in \mathrm{Topk}(\{s_{j,t} | 1 \leqslant j \leqslant N_r\}, K_r))$：专家 i 的负载频率
- $s'_{i,t} = \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}}$：归一化亲和力分数
- $P_i = \frac{1}{T} \sum_{t=1}^{T} s'_{i,t}$：专家 i 的平均亲和力
- α：均衡因子（在DeepSeek-V3中赋值为很小的正数）

1.3.2 设计意图

- 鼓励单个序列内专家负载的均衡分布
- 与主损失函数协同工作，避免对模型性能产生负面影响

2 多token预测训练目标

2.1 动机与理论基础

多token预测（MTP）目标的引入基于双重考虑：

- **数据效率提升**：使训练信号更加密集，提高每个训练样本的利用率
- **表示学习增强**：使模型能够预先规划其表示形式，更好地预测未来token

2.2 与现有方法的区别

与Gloeckle等人（2024）的并行预测方法不同，DeepSeek采用**顺序预测架构**：

- **保留因果链**：在每个预测深度维持完整的因果依赖关系
- **渐进式预测**：通过多个顺序模块逐步预测未来token

2.3 架构实现细节

2.3.1 MTP模块组成

对于第 k 个预测模块（$k = 1, \ldots, D$）：

- **共享组件**：
  - 嵌入层 $\mathrm{Emb}(\cdot)$（与主模型共享）
  - 输出头 $\mathrm{OutHead}(\cdot)$（与主模型共享）
- **专用组件**：
  - Transformer块 $\mathrm{TRM}_k(\cdot)$
  - 投影矩阵 $M_k \in \mathbb{R}^{d \times 2d}$

2.3.2 前向传播过程

对于第 i 个输入token t_i，在第 k 个预测深度：

1. **输入拼接与投影**：

$$\mathbf{h}_i'^k = M_k[\mathrm{RMSNorm}(\mathbf{h}_i^{k-1}); \mathrm{RMSNorm}(\mathrm{Emb}(t_{i+k}))]$$

   其中：
   - $\mathbf{h}_i^{k-1}$：来自前一个预测深度的表示（当 $k = 1$ 时来自主模型）
   - $[\cdot;\cdot]$：拼接操作
2. **Transformer处理**：

$$\mathbf{h}_{1:T-k}^k = \mathrm{TRM}_k(\mathbf{h}_{1:T-k}'^k)$$

   - T：输入序列长度
   - $\mathbf{h}_{i:j}$：表示切分操作（包含左右边界）
3. **概率预测**：

$$P_{i+k+1}^k = \mathrm{OutHead}(\mathbf{h}_i^k)$$

   - 输出头将表示线性映射为logits，再经Softmax得到概率分布

- V：词表大小

2.4 损失计算与训练目标

2.4.1 各深度损失

对于每个预测深度 k，计算交叉熵损失：

$$\mathcal{L}_{\text{MTP}}^{k} = \text{CrossEntropy}(P_{2+k:T+1}^{k}, t_{2+k:T+1}) = -\frac{1}{T}\sum_{i=2+k}^{T+1}\log P_i^k[t_i]$$

其中：

- t_i：第 i 个位置的ground-truth token
- $P_i^k[t_i]$：第 k 个MTP模块对 t_i 的预测概率

2.4.2 总MTP损失

$$\mathcal{L}_{\text{MTP}} = \frac{\lambda}{D}\sum_{k=1}^{D}\mathcal{L}_{\text{MTP}}^{k}$$

- λ：权重因子，控制MTP损失在总损失中的比重
- D：预测深度（额外预测的token数量）

2.5 推理时应用

2.5.1 标准推理模式

- **模块丢弃**：MTP模块在推理时可完全移除
- **独立工作**：主模型可正常独立运行，不受MTP影响

2.5.2 推测解码应用

- **可选功能**：保留MTP模块用于推测解码
- **延迟优化**：通过预测多个token来降低生成延迟
- **与EAGLE的区别**：虽然原理类似，但主要目标不同（训练增强 vs. 推理加速）

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

1 理论动机：为何要探索“纯 RL”驱动推理?

1.1 当前后训练范式的瓶颈

当前主流 LLM 后训练流程通常遵循：预训练 → SFT（监督微调） → 奖励建模（RM） → 强化学习（RLHF/RLAIF）

其中，SFT 是不可或缺的“冷启动”步骤，依赖大量高质量标注数据（如人工编写的 CoT）。然而：

- **数据获取成本高**：优质长 CoT 数据需专家撰写或复杂合成。
- **泛化能力受限**：SFT 数据分布决定了模型上限，难以超越训练集覆盖范围。
- **策略固化风险**：模型容易模仿固定解题模板，缺乏自主探索能力。

1.2 强化学习作为“智能涌现引擎”的潜力

DeepSeek 团队提出一个根本性假设：

> **LLM 具备内在的推理潜力，只需通过合适的奖励信号引导，即可在无监督条件下自我演化出高效的问题解决策略。**

这一假设源于认知科学中的“试错学习”理论——人类智能并非全靠教导，而是在反馈中不断调整行为。若能在 LLM 中复现此过程，则可突破 SFT 的天花板。

2 DeepSeek-R1-Zero：纯强化学习的技术实现细节

2.1 基础模型选择：DeepSeek-V3-Base

- **未经过任何指令微调**，保留原始语言建模能力。
- 参数量级为百亿级别，具备足够容量支持复杂推理。
- 选择基础模型而非 Instruct 模型，是为了避免先验偏置干扰 RL 的自然探索。

2.2 奖励函数设计：规则驱动 vs 神经网络 RM

2.2.1 为何放弃神经网络奖励模型?

尽管 RM 在对齐任务中广泛应用，但在纯推理场景存在三大问题：

问题	说明
奖励黑客（Reward Hacking）	模型学会欺骗 RM（如生成看似合理但错误的 CoT）
评估滞后	RM 需持续更新以匹配策略模型，增加训练复杂度
泛化偏差	RM 在训练分布外问题上不可靠（如新数学定理）

因此，DeepSeek 采用**完全确定性的规则奖励系统**：

2.2.2 准确性奖励（Accuracy Reward）

- **数学题**：
  - 要求最终答案格式为 `\boxed{...}`。
  - 使用 SymPy 或数值验证器判断等价性（如 `2+2=4` 与 `4` 视为等价）。
  - 对开放题（如证明题），采用多步验证：中间结论是否逻辑自洽。
- **编程题**（LeetCode 风格）：
  - 编译代码并运行预设测试用例。
  - 奖励 = 通过测试数 / 总测试数。
  - 支持超时与内存限制，防止无限循环。

2.2.3 格式奖励（Format Reward）

- 强制结构：`<think> ... </think>` 包裹推理过程。
- 若缺失标签或格式错误，奖励直接归零。
- 此设计确保后续可解析 CoT，便于拒绝采样与蒸馏。

2.2.4 语言一致性奖励（仅用于 DeepSeek-R1）

- 计算 CoT 中非目标语言字符比例（如中文提示下出现英文段落）。
- 奖励 = 1 - α × (非目标语言比例)，α 为调节系数（实验设为 0.5）。
- 实验表明：该奖励使语言混合率下降 80%，可读性显著提升。

2.3 提示模板设计：最小干预原则

为避免引入人为偏置，提示模板极为简洁：

- **不指定解题方法**（如“使用归纳法”）。
- **不强制反思**（如“检查你的答案”）。
- 目标：让模型在 RL 过程中**自主发现**最佳策略。

3 DeepSeek-R1：四阶段训练管线详解

3.1 阶段 1：冷启动 SFT（Cold Start SFT）

3.1.1 数据构建策略

来源	数量	特点
Few-shot 提示生成	~2000	使用 o1 风格提示引导长 CoT
R1-Zero 优质输出	~1500	人工筛选正确且结构清晰的响应
人类标注优化	~500	修正语言、添加总结、统一格式

3.1.2 格式标准化

采用双分隔符结构：

```
<sot><reasoning>...</reasoning><eot><summary>...</summary>
```

- `<sot>/<eot>` 为特殊 token，便于模型学习边界。
- `<summary>` 必须简洁、独立可读，不含技术细节。

3.2 阶段 2：推理聚焦 RL

- **训练数据分布**：90% 推理任务（数学/代码/逻辑），10% 通用任务。
- **批大小**：8192（极大 batch 提升梯度稳定性）。
- **KL 散度约束**：防止策略偏离初始模型过远（KL coeff = 0.01）。
- **收敛判断**：AIME 验证集 pass@1 连续 500 步波动 $< 0.5\%$。

3.3 阶段 3：拒绝采样 + 多领域 SFT

3.3.1 推理数据生成流程

1. 从 RL 检查点采样 16 个响应/提示。
2. 应用规则过滤器：
    - 语言一致性 $\geq 90\%$
    - 无冗余重复段落
    - CoT 长度 $\in [200, 2000]$ tokens
3. 对无法规则验证的问题（如开放证明），使用 **DeepSeek-V3 作为裁判模型**：
    - 输入：(问题, 标准答案, 模型响应)
    - 输出：是否正确（二分类）
    - 该过程称为“生成式奖励建模”，但仅用于数据筛选，不参与 RL。

3.3.2 非推理数据处理

- 对写作、角色扮演等任务，**有条件生成 CoT**：
    - 复杂查询（如“写一篇关于量子计算的科普文章”）$\rightarrow$ 生成大纲式 CoT。
    - 简单查询（如“今天天气如何？”）$\rightarrow$ 直接回答，避免冗余。
- 此设计平衡了“推理普适性”与“响应效率”。

3.4 阶段 4：全场景 RL

3.4.1 奖励信号融合

任务类型	奖励来源	评估对象
数学/代码	规则奖励	完整响应（含 CoT）
写作/QA	神经 RM	仅 `<summary>`
安全敏感	安全 RM	完整响应

- **有用性 RM**：基于 DeepSeek-V3 的偏好数据训练，关注信息量、相关性。
- **无害性 RM**：检测偏见、歧视、非法建议等，覆盖 CoT 中的潜在风险。

3.4.2 训练技巧

- **课程学习**（Curriculum Learning）：初期侧重推理任务，后期逐步增加通用任务比例。
- **动态权重调整**：根据验证集表现自动平衡各奖励项权重。

DeepSeek-V3.2-Exp: Boosting Long-Context Efficiency with DeepSeek Sparse Attention

1 研究背景：长上下文 LLM 的效率瓶颈

在大语言模型（LLM）持续扩展上下文窗口至 128K 甚至更高时，**稠密注意力机制**（dense attention）的计算与内存开销成为主要瓶颈。其核心问题在于：

- **计算复杂度**为 $O(L^2)$，其中 L 为序列长度；
- 在自回归生成（decoding）阶段，KV Cache 随上下文线性增长，导致显存爆炸；
- 即使使用 MLA（Multi-head Latent Attention）等压缩技术，长上下文下的注意力仍占据推理成本主导地位。

因此，一个**可训练、细粒度、硬件友好的稀疏注意力机制**成为提升长上下文效率的关键。DeepSeek-V3.2-Exp 通过引入 **DeepSeek Sparse Attention**（DSA） 正是对这一挑战的系统性回应。

2 核心创新：DeepSeek Sparse Attention（DSA）的深层设计逻辑

2.1 常规做法 vs DeepSeek 的选择

- **常规稀疏注意力**（如 Local + Global、Fixed Pattern）：
  - 通常采用**静态模式**（如 sliding window、block diagonal）；
  - 无法适应不同 token 的动态语义依赖；
  - 难以与高性能推理框架（如 vLLM）高效集成。
- **DeepSeek 的选择**：
  - 设计一个**可训练、动态、轻量级的 token 重要性预测器**（Lightning Indexer）；
  - 基于预测结果，**为每个查询 token 独立选择 Top-k 最相关的历史 token**；
  - 整体架构**兼容 MLA 与 MQA**，实现训练-推理一体化。

理论依据：并非所有历史 token 对当前预测都同等重要。通过**学习“注意力先验”**，可显著减少冗余计算，同时保留关键依赖。

3 DSA 技术细节：两大组件协同工作

3.1 1. Lightning Indexer（闪电索引器）—— 动态相关性预测

功能：为查询 token h_t 与每个历史 token h_s 计算一个轻量级“相关性分数” $I_{t,s}$。

数学形式：

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \mathrm{ReLU}\left(q_{t,j}^I \cdot k_s^I\right)$$

- H^I：索引头数量（通常 ≤ 8），远小于主模型头数；
- $q_{t,j}^I, w_{t,j}^I$ 由 h_t 通过小型线性投影生成；
- k_s^I 由 h_s 生成；
- **激活函数选择 ReLU**：避免 Softmax 的全局归一化开销，提升吞吐；
- **支持 FP8 实现**：计算量极小（$< 5\%$ 总 FLOPs），对性能影响可忽略。

关键设计动机：用一个**低秩、低精度、可微分的轻量网络**，近似主注意力的跨头聚合行为。

3.2 2. Fine-grained Top-k Token Selection（细粒度 Top-k 选择）

流程：

1. 对每个查询 token h_t，计算其与所有历史 token 的 $I_{t,s}$；
2. 选择分数最高的 k 个 token（如 $k = 2048$）；
3. 仅对这 k 个 token 的 latent key-value 向量 c_s 执行主注意力计算：

$$u_t = \mathrm{Attn}\left(h_t,\ \{c_s \mid I_{t,s} \in \text{Top-}k(I_{t,:})\}\right)$$

- **主注意力基于 MLA 架构**，其 c_s 是压缩后的 KV 表示；
- **复杂度从 $O(L^2)$ 降至 $O(Lk)$**，在 $L = 128K, k = 2K$ 时理论加速 64 倍。

工程意义：该选择机制天然适配 **Multi-Query Attention**（MQA） 模式，每个 token 共享 latent KV，便于 PagedAttention 等高效缓存策略。

4 与 MLA 架构的融合：为何选择 MQA 模式?

DeepSeek-V3.1-Terminus 使用 **MLA**（Multi-head Latent Attention），其 KV 以 latent 向量形式存储。DSA 在其基础上进一步优化：

- **训练阶段**：使用 MLA-MHA 模式（多头）以保留表达能力；
- **推理阶段**（含 DSA）：切换至 MLA-MQA 模式（单 KV 头，多 Query 头）；
- **原因**：
  - MQA 模式下，每个 latent KV 向量被所有 query heads 共享；
  - 显著减少 KV Cache 大小（从 $H \times L \to 1 \times L$）；
  - 与 Top-k 选择机制天然契合：只需为每个 token 选 k 个 latent 向量，而非 $H \times k$。

图示参考：论文图1清晰展示了 DSA 如何在 MLA-MQA 框架中插入 Indexer 与 Top-k Selector。

5 训练策略：两阶段持续训练确保稳定性

5.1 阶段1：Dense Warm-up（稠密预热）

- **目标**：让 Lightning Indexer 学会预测主注意力的分布；
- **方法**：
  - 冻结主模型，仅训练 Indexer；
  - 主注意力跨头求和后 L1 归一化，得到目标分布 $p_{t,:}$；
  - 使用 **KL 散度损失**：

$$\mathcal{L}_I = \sum_t D_{\mathrm{KL}}(p_{t,:} \,\|\, \mathrm{Softmax}(I_{t,:}))$$

- **数据**：1000 步 $\times$ 16 $\times$ 128K = **2.1B tokens**，LR = 10^{-3}。

5.2 阶段2：Sparse Training（稀疏训练）

- **引入 Top-k 选择**，端到端训练；
- **Indexer 与主模型解耦优化**：
  - Indexer 仍用 KL loss，但**仅在 Top-k 子集上计算**（公式4）；
  - 主模型仅用 **语言建模 loss**；
  - Indexer 输入从计算图中 **detach**，避免梯度干扰。
- **数据**：15000 步 $\times$ 480 $\times$ 128K $=$ **943.7B tokens**，$k = 2048$，LR $= \mathbf{7.3 \times 10^{-6}}$。

关键洞察：解耦训练是稳定稀疏注意力的核心——indexer 负责“选对”，主模型负责“用好”。

DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning

1 研究背景：超越“答对即正确”的推理瓶颈

大语言模型（LLM）在数学推理领域取得了显著进展，尤其在**以最终答案为评判标准的数值型竞赛**（如 AIME、HMMT）中，通过强化学习（RL）已接近性能饱和。然而，这一范式存在**两个根本性局限**：

1. **正确答案 ≠ 正确推理**：模型可能通过错误逻辑、巧合甚至“答案背诵”获得正确结果，其推理过程不可信、不可复现。
2. **不适用于定理证明**：许多高阶数学任务（如 IMO 几何、不等式证明）不要求数值答案，而要求**严谨的自然语言推导**，此时“最终答案奖励”完全失效。

DeepSeek 的洞察：要真正推进深度数学推理，必须让模型具备**自我验证能力**——即在无标准答案的情况下，判断自身证明是否**逻辑自洽、步骤完整、推导严谨**。这不仅是性能提升的关键，更是迈向**科研级数学 AI**的必经之路。

2 核心框架：生成器–验证器协同进化（Generator–Verifier Co-evolution）

DeepSeekMath-V2 的核心创新在于构建了一个**闭环协同系统**，由两个深度耦合的 LLM 组件构成：**Proof Verifier**（证明验证器） 与 **Proof Generator**（证明生成器）。二者通过**奖励机制与训练目标**紧密联动，形成正反馈循环。

2.1 Proof Verifier：忠实、可信赖的逻辑审计员

2.1.1 功能定义

- **输入**：问题 X + 证明 Y
- **输出**：
  - 对 Y 中逻辑缺陷的**详细分析**；
  - 一个 **三档评分**：
    - **1**：完全严谨（complete and rigorous）
    - **0.5**：整体正确但有细节瑕疵
    - **0**：存在致命逻辑错误

2.1.2 训练挑战：幻觉式缺陷（Hallucinated Issues）

- 传统 RL 仅监督 **评分准确性**，不监督 **缺陷分析的真实性**；
- 导致 Verifier 在低分证明上“编造理由”以获得高奖励，**破坏可信度**。

2.1.3 解法：引入 Meta-Verifier（元验证器）

- **动机**：评估 Verifier 的分析是否**真实、合理、有依据**。
- **训练流程**：
  1. 用初始 Verifier 生成分析 V_i；
  2. **专家标注分析质量** $ms_i \in \{0, 0.5, 1\}$，形成数据集 D_{mv}；
  3. 训练 Meta-Verifier π_η，输出对 V_i 的评估与质量分；
  4. **增强 Verifier 训练**：将 Meta-评分 R_{meta} 纳入奖励：

$$R_V = R_{\text{format}} \cdot R_{\text{score}} \cdot R_{\text{meta}}$$

效果：Verifier 分析的 **平均 Meta-评分从 0.85 提升至 0.96，分析真实性大幅提升**，而评分准确率保持不变。

2.2 Proof Generator：具备自省能力的证明生成器

2.2.1 训练目标

- 不仅生成证明 Y，还**自我分析并评分** Z，格式与 Verifier 一致；
- 通过 Verifier 与 Meta-Verifier **联合评估** Y **与** Z。

2.2.2 奖励函数设计（关键创新）

$$R = R_{\text{format}}(Y, Z) \cdot (\alpha \cdot R_Y + \beta \cdot R_Z)$$

其中：

- $R_Y = s$：Verifier 对 Y 的客观评分；
- $R_Z = R_{\text{score}}(s', s) \cdot R_{\text{meta}}(Z)$：对自我分析 Z 的评估；
- $\alpha = 0.76$, $\beta = 0.24$：平衡生成质量与自省能力。

2.2.3 激励机制

场景	奖励
正确证明 + 准确自评（score=1）	**最高**
错误证明 + **诚实自评**（score=0）	**中等**
错误证明 + **谎称正确**（score=1）	**极低**

本质：模型学会“**先自我批判，再输出**”，模拟人类数学家的审慎思维，**内化验证能力**。

3 自动化训练闭环：Scaling Verification Compute（扩展验证计算）

随着 Generator 能力提升，它会生成**越来越难验证的新证明**（Hard Negatives），这些样本成为 Verifier 的新训练数据。为避免人工标注瓶颈，作者设计了一个**全自动标注流水线**，实现**无需人工干预的持续进化**。

3.1 自动标注流程（Fully Automated Labeling）

1. **多视角验证**：
   - 对每个新证明 Y，运行 $n = 64$ **次独立 Verifier 分析**；
2. **Meta-验证确认缺陷**：
   - 对每个评分 ≤ 0.5 的分析，运行 $m = 8$ **次 Meta-Verifier 评估**；
   - 若多数 Meta-Verifier 确认缺陷存在 $\rightarrow$ 该分析**有效**；
3. **标签决策规则**：
   - 若存在 **至少** k **个有效低分分析** $\rightarrow$ 证明标签为该最低分；
   - 若**所有分析均未发现有效缺陷** $\rightarrow$ 证明标签为 **1**；
   - 否则 $\rightarrow$ 送人工审核（实际后期几乎无需）。

3.2 为什么可行?

- **Scaling Verifier Samples**：增加采样次数显著提升**发现真实缺陷的概率**；
- **Meta-Verification 更易学**：相比从零发现缺陷，**判断已有分析是否合理**对 LLM 更简单、更样本高效；
- **生成–验证能力差**（Generation–Verification Gap）：更强的 Generator 自然产生挑战 Verifier 的 Hard Proofs，推动其升级。

闭环本质：**"生成 → 验证 → 自省 → 自动标注 → 再训练"** 的正反馈循环，使系统在**无标准答案**的定理证明任务上持续进步。

3.3 实际效果

- **最后两轮训练完全无需人工标注**；
- 自动标签与专家判断**高度一致**；
- 在 **IMO 2025、CMO 2024** 上达到**人类金牌水平**；
- 在 **Putnam 2024** 上取得 **118/120 分**，**超越人类最高分（90）**。